Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра интеллектуальных информационных технологий

Лабораторная работа №4

по дисциплине «Модели решения задач в интеллектуальных системах»

За седьмой семестр

Тема:

«Многослойный персептрон для прогнозирования временного ряда: влияние размера окна и числа скрытых нейронов»

Выполнил:
студент 4 курса
группы ИИ-26
Ковальчук А.И.
Проверил:
Никитин С. К.

Брест 2026

1 Цель работы и постановка задачи

Цель работы — на основе многослойного персептрона архитектуры p–N–1 решить задачу прогнозирования временного ряда; научиться формировать обучающую выборку методом скользящего окна, нормализовать данные, оценивать качество одношагового и многошагового прогноза и исследовать влияние размера окна p и числа нейронов скрытого слоя N на точность прогноза и скорость обучения.

Работа является продолжением лабораторных работ № 1–3, в которых на той же программной реализации сети (классы DenseLayer, Sequential) исследовались разделённость XOR, онлайн-обучение и выбор функции активации. Вариант задания сохранён: № 7.

Прогнозируемый ряд задаётся формулой

$$x_k = a \cdot \cos(b \cdot k) + c \cdot \sin(d \cdot k), \quad k = 0, 1, \ldots, 199. \tag{1}$$

Параметры варианта 7: a = 3,0; b = 0,22; c = 1,5; d = 0,55; p = 8; N = 6. Первые 70 % точек (k = 0…139) образуют обучающий участок, остальные 30 % (k = 140…199) — тестовый. Сеть по p предыдущим значениям предсказывает следующее:

$$\hat{x}_k = F(x_{k-1}, x_{k-2}, \ldots, x_{k-p}). \tag{2}$$

2 Описание метода

2.1 Нормализация

Минимум и максимум вычисляются только по обучающему участку. Нормализация и обратный пересчёт:

$$x_norm = (x - x_min) / (x_max - x_min), \quad (3)$$

$$x = x_norm \cdot (x_max - x_min) + x_min. \quad (4)$$

Min/max нельзя брать по всему ряду: иначе в оценку масштаба «утекает» тестовый участок, и сравнение train/test перестаёт быть честной оценкой обобщающей способности. Если тестовые значения выходят за диапазон обучения, после (3) они оказываются вне отрезка [0; 1]. Линейный выход это допускает, сигмоида — нет.

2.2 Скользящее окно

Для каждого k = p, p+1, …, 199 формируется пример: вход — вектор (x_{k-p}, …, x_{k-1}), цель — x_k. Обучающая выборка содержит цели k = p…139 (140 − p примеров), тестовая — k = 140…199 (60 примеров). При p = 8 это 132 и 60 примеров соответственно.

2.3 Архитектура p–N–1 и градиенты

Скрытый слой — сигмоида σ(s) = 1 / (1 + e^{−s}), выходной — линейный f(s) = s (задача регрессии). Функция потерь на одном примере и средняя ошибка эпохи:

$$E = ½ (y - d)^2, \quad E_s = (1/L) \Sigma_{i=1}^{L} (y_i - d_i)^2. \quad (5)$$

Обучение — алгоритм обратного распространения в онлайн-режиме (batch_size = 1) с перемешиванием порядка примеров в каждой эпохе. Скорость обучения η = 0,1 выбрана из диапазона 0,01–0,3: при 0,01 сходимость чрезмерно медленная,

при 0,3 обновления сигмоиды становятся менее устойчивыми. Критерий остановки: E_s ≤ E_e = 10^{−4} (нормализованная шкала) либо 20 000 эпох.

Прямой проход для скрытого нейрона j и линейного выхода:

$$s_j = \Sigma_{i=1}^{p} x_i w_{ij} + b_j, \quad h_j = \sigma(s_j), \tag{6}$$

$$y = \Sigma_{j=1}^{N} h_j w_j^{out} + b^{out}. \tag{7}$$

Градиенты для линейного выхода (∂E/∂y = y − d) и сигмоидного скрытого слоя:

$$\delta^{out} = y - d, \tag{8}$$

$$\delta_j^h = \delta^{out} \cdot w_j^{out} \cdot h_j (1 - h_j), \tag{9}$$

$$\Delta w_j^{out} = -\eta \delta^{out} h_j, \quad \Delta b^{out} = -\eta \delta^{out}, \tag{10}$$

$$\Delta w_{ij} = -\eta \delta_j^h x_i, \quad \Delta b_j = -\eta \delta_j^h. \tag{11}$$

Формула (9) есть произведение ошибки, пришедшей с выхода, на производную сигмоиды σ'(s) = σ(s)(1 − σ(s)). Для линейного выхода производная равна 1, поэтому δ^{out} совпадает с (y − d).

2.4 Метрики качества

Все метрики считаются после обратного пересчёта (4) в исходной шкале:

$$MAE = (1/n) \Sigma |y_i - \hat{y}_i|, \quad RMSE = \sqrt{[(1/n) \Sigma (y_i - \hat{y}_i)^2]}. \tag{12}$$

Ряд (1) осциллирует около нуля и многократно меняет знак. MAPE содержит деление на y_i и при y_i ≈ 0 даёт выбросы. Поэтому вместо MAPE используется симметричная мера

$$sMAPE = (100\,\% / n)\,\Sigma\; 2\,|y_i - \hat{y}_i| \,/\, (|y_i| + |\hat{y}_i|). \qquad (13)$$

2.5 Рекурсивный многошаговый прогноз

Начиная с k = 140 окно заполняется истинными значениями x_{140−p}…x_{139}. Далее каждое предсказание ŷ подставляется в окно как самое новое значение, и процедура повторяется на 30 шагов. Ошибка накапливается, потому что на вход следующего шага подаётся уже искажённый вектор. Горизонт считается пригодным, пока абсолютная ошибка не превышает 5 % размаха ряда.

3 Графическая схема пайплайна

На рисунке 1 приведена цепочка обработки данных. Набор блоков выбран так, чтобы он совпадал с постановкой лабораторной и не содержал лишних стадий (регуляризация, dropout, пакетная нормализация, ранняя остановка по валидации).

Графическая схема пайплайна обработки данных

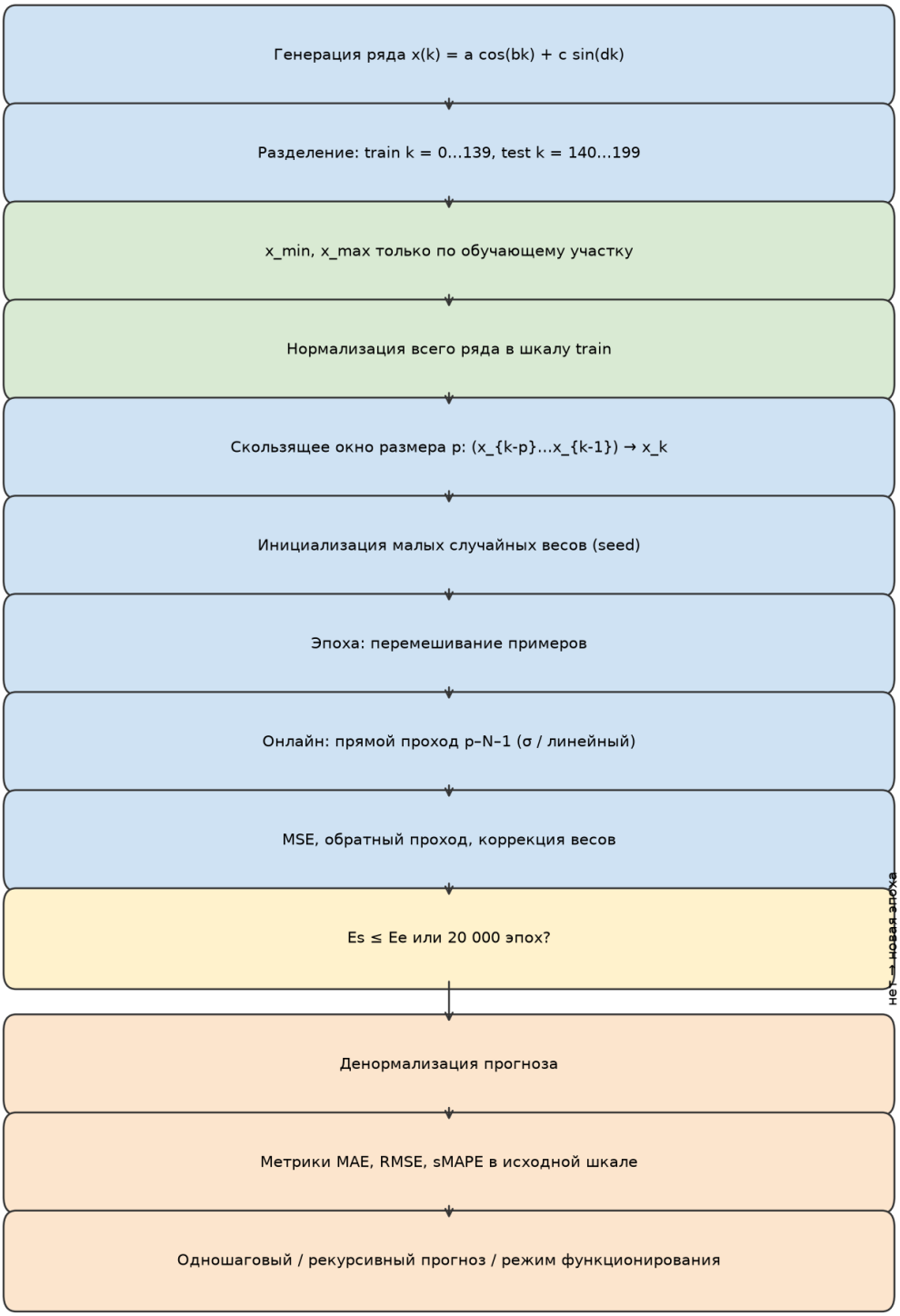


Рисунок 1 – Графическая схема пайплайна обработки данных

Пояснение выбора блоков. «Генерация ряда» задаёт объект исследования. «Разделение train/test» выполняется до

нормализации, чтобы граница k = 140 была фиксированной во времени, а не зависела от масштаба. Блок «x_min, x_max только по train» обязателен: именно он запрещает утечку теста в масштаб. «Скользящее окно» превращает ряд в выборку пар (X, d) для сети p–N–1. Цикл «перемешивание — прямой проход — MSE — обратный проход» реализует онлайн-обучение с критерием E_s. Жёлтый ромб — развилка остановки. Оранжевые блоки относятся уже к функционированию обученной сети: денормализация, метрики, одношаговый и рекурсивный прогнозы, режим ввода пользовательского окна. Такая цепочка достаточна и минимальна: каждый блок либо готовит данные, либо обучает модель, либо оценивает прогноз.

4 Ручной расчёт одного шага обучения

Рассмотрим сеть 3–2–1 с параметрами, привязанными к варианту V = 7. Входы X = [0,2; 0,5; 0,8], цель d = 0,6, скорость η = 0,1. Веса скрытого слоя и смещения:

$$w_{11} = 0{,}05 \cdot V = 0{,}35, \quad w_{12} = -0{,}2, \tag{14}$$

$$w_{21} = 0{,}3, \quad w_{22} = 0{,}05 \cdot V = 0{,}35, \tag{15}$$

$$w_{31} = -0{,}1, \quad w_{32} = 0{,}4, \quad b_1 = 0{,}1, \quad b_2 = -0{,}1. \tag{16}$$

Веса выходного слоя:

$$w_1^{out} = 0{,}5, \quad w_2^{out} = -0{,}2 \cdot V = -1{,}4, \quad b^{out} = 0{,}2. \tag{17}$$

4.1 Прямой проход

Взвешенные суммы скрытых нейронов:

$$s_1 = 0{,}2\cdot 0{,}35 + 0{,}5\cdot 0{,}3 + 0{,}8\cdot(-0{,}1) + 0{,}1 = 0{,}24, \quad (18)$$

$$s_2 = 0{,}2\cdot(-0{,}2) + 0{,}5\cdot 0{,}35 + 0{,}8\cdot 0{,}4 - 0{,}1 = 0{,}355. \quad (19)$$

Активации скрытого слоя:

$$h_1 = \sigma(0{,}24) = 1/(1 + e^{-0{,}24}) = 0{,}559714, \quad (20)$$

$$h_2 = \sigma(0{,}355) = 1/(1 + e^{-0{,}355}) = 0{,}587830. \quad (21)$$

Линейный выход и значение функции потерь:

$$y = 0{,}559714\cdot 0{,}5 + 0{,}587830\cdot(-1{,}4) + 0{,}2 = -0{,}343105, \quad (22)$$

$$E = \tfrac{1}{2}(y - d)^2 = \tfrac{1}{2}(-0{,}343105 - 0{,}6)^2 = 0{,}444723. \quad (23)$$

4.2 Обратный проход

Ошибка выходного слоя и ошибки скрытых нейронов по (8)–(9):

$$\delta^{out} = y - d = -0{,}943105, \quad (24)$$

$$\delta_1^h = (-0{,}943105)\cdot 0{,}5\cdot 0{,}559714\cdot(1 - 0{,}559714) = -0{,}116207, \quad (25)$$

$$\delta_2^h = (-0{,}943105)\cdot(-1{,}4)\cdot 0{,}587830\cdot(1 - 0{,}587830) = 0{,}319901. \quad (26)$$

Обновление выходного слоя по (10):

$$w_1^{out} \leftarrow 0{,}5 - 0{,}1\cdot(-0{,}943105)\cdot 0{,}559714 = 0{,}552787, \quad (27)$$

$$w_2^{out} \leftarrow -1{,}4 - 0{,}1\cdot(-0{,}943105)\cdot 0{,}587830 = -1{,}344562, \quad (28)$$

$$b^{out} \leftarrow 0{,}2 - 0{,}1\cdot(-0{,}943105) = 0{,}294311. \quad (29)$$

Обновление скрытого слоя по (11):

$$w_{11} \leftarrow 0{,}35 - 0{,}1\cdot(-0{,}116207)\cdot 0{,}2 = 0{,}352324, \quad (30)$$

$$w_{12} \leftarrow -0{,}2 - 0{,}1\cdot 0{,}319901\cdot 0{,}2 = -0{,}206398, \quad (31)$$

$$w_{21} \leftarrow 0{,}30 - 0{,}1\cdot(-0{,}116207)\cdot 0{,}5 = 0{,}305810, \quad (32)$$

$$w_{22} \leftarrow 0{,}35 - 0{,}1 \cdot 0{,}319901 \cdot 0{,}5 = 0{,}334005, \quad (33)$$

$$w_{31} \leftarrow -0{,}1 - 0{,}1 \cdot (-0{,}116207) \cdot 0{,}8 = -0{,}090703, \quad (34)$$

$$w_{32} \leftarrow 0{,}4 - 0{,}1 \cdot 0{,}319901 \cdot 0{,}8 = 0{,}374408, \quad (35)$$

$$b_1 \leftarrow 0{,}1 - 0{,}1 \cdot (-0{,}116207) = 0{,}111621, \quad (36)$$

$$b_2 \leftarrow -0{,}1 - 0{,}1 \cdot 0{,}319901 = -0{,}131990. \quad (37)$$

Численные значения совпадают с контрольным прогоном той же сети в программе (ячейка проверки ручного шага). Знак δ^{out} отрицательный, потому что сеть выдала $y < d$; веса, ведущие к увеличению y, возрастают.

5 Программная реализация

Код выполнен в Jupyter-ноутбуке src/lab.ipynb на той же реализации сети, что в лабораторных работах № 1–3: классы DenseLayer, Input, Dropout и Sequential не переписывались. Изменены только расчёт средней ошибки эпохи $E_s = \text{mean}((y - d)^2)$ и частота печати. Запрещённые фреймворки (scikit-learn, PyTorch, TensorFlow) не использовались; окно и метрики реализованы на NumPy.

Структура программы соответствует требованию «отдельные функции»: generate_series, normalize / denormalize, make_windows, build_mlp, train_one, mae / rmse / smape, recursive_forecast, functioning_mode. Веса инициализируются равномерно на [−0,5; 0,5], воспроизводимость задаётся seed. Полный листинг приведён в приложении А и в файле src/lab.ipynb.

```
VARIANT = 7
A, B, C, D = 3.0, 0.22, 1.5, 0.55
P_VAR, N_VAR = 8, 6

K_TOTAL = 200
TRAIN_END = 140
EE = 1e-4
MAX_EPOCHS = 20000
ALPHA = 0.1
BATCH_SIZE = 1
SEEDS = [42, 7, 13, 21, 99]
HORIZON = 30

PLOTS_DIR = Path.cwd() / "plots"
PLOTS_DIR.mkdir(exist_ok=True)


def generate_series(a=A, b=B, c=C, d=D, n=K_TOTAL):
    k = np.arange(n)
    return a * np.cos(b * k) + c * np.sin(d * k)


def normalize(x, x_min, x_max):
    return (x - x_min) / (x_max - x_min)


def denormalize(x_norm, x_min, x_max):
    return x_norm * (x_max - x_min) + x_min
```

```
def make_windows(series, p):
    """Скользящее окно: вход — p предыдущих значений, цель — следующее."""
    X, y, ks = [], [], []
    for k in range(p, len(series)):
        X.append(series[k - p:k])
        y.append(series[k])
        ks.append(k)
    return np.asarray(X, dtype=np.float32), np.asarray(y, dtype=np.float32), np.asarray(ks)


def mae(y, y_hat):
    y, y_hat = np.asarray(y).reshape(-1), np.asarray(y_hat).reshape(-1)
    return float(np.mean(np.abs(y - y_hat)))


def rmse(y, y_hat):
    y, y_hat = np.asarray(y).reshape(-1), np.asarray(y_hat).reshape(-1)
    return float(np.sqrt(np.mean((y - y_hat) ** 2)))


def mape(y, y_hat):
    y, y_hat = np.asarray(y).reshape(-1), np.asarray(y_hat).reshape(-1)
    return float(np.mean(np.abs((y - y_hat) / y)) * 100.0)


def smape(y, y_hat):
    y, y_hat = np.asarray(y).reshape(-1), np.asarray(y_hat).reshape(-1)
    denom = np.abs(y) + np.abs(y_hat)
```

```
    return float(np.mean(2.0 * np.abs(y - y_hat) / np.where(denom == 0, 1.0,
denom)) * 100.0)


def metrics_orig(y_true, y_pred):
    return {"MAE": mae(y_true, y_pred), "RMSE": rmse(y_true, y_pred),
"sMAPE": smape(y_true, y_pred)}


def build_mlp(p, n, seed=None):
    if seed is not None:
        np.random.seed(seed)
    model = Sequential([
        Input(),
        DenseLayer(units=n, activation="sigmoid"),
        DenseLayer(units=1, activation="linear"),
    ])
    fan_in = p
    for layer in model.layers:
        if isinstance(layer, DenseLayer):
            layer.init_weights(fan_in)
            fan_in = layer.units
    model.compile(loss="mse")
    return model


def train_one(X, y, p, n, seed, alpha=ALPHA, ee=EE, epochs=MAX_EPOCHS,
verbose=False):
    model = build_mlp(p, n, seed=seed)
    n_epochs, converged = model.fit(
```

```
            X, y, epochs=epochs, alpha=alpha, batch_size=BATCH_SIZE, Ee=ee,
verbose=verbose,
        )
        return {
            "model": model,
            "epochs": n_epochs,
            "converged": converged,
            "Es": float(model.history[-1]),
            "history": list(model.history),
            "seed": seed,
            "p": p,
            "N": n,
        }


    def evaluate_split(model, X, y, x_min, x_max):
        pred_n = model.predict(X).reshape(-1)
        pred = denormalize(pred_n, x_min, x_max)
        true = denormalize(np.asarray(y).reshape(-1), x_min, x_max)
        return pred, true, metrics_orig(true, pred)


    def recursive_forecast(model, series_norm, p, start_k, steps):
        window = list(series_norm[start_k - p:start_k])
        preds = []
        for _ in range(steps):
            y_hat = float(model.predict(np.array(window, dtype=np.float32))[0, 0])
            preds.append(y_hat)
            window = window[1:] + [y_hat]
        return np.array(preds)
```

```
        def functioning_mode(model, p, x_min, x_max, window=None, k=None,
series=None, label=""):
            if window is None:
                window = series[k - p:k]
            window = np.asarray(window, dtype=float)
            true = float(series[k]) if (k is not None and series is not None) else None

            wn = normalize(window, x_min, x_max)
            y_n = float(model.predict(wn.astype(np.float32))[0, 0])
            y_hat = denormalize(y_n, x_min, x_max)

            head = label if label else "окно"
            if k is not None:
                head += f", k={k}"
            print(f"[{head}]")
            print("  окно (исх. шкала):    ", np.array2string(window, precision=4,
separator=", "))
            print("  нормализованные входы:", np.array2string(wn, precision=4,
separator=", "))
            if np.any((wn < 0) | (wn > 1)):
                print("  (!) часть входов вне [0; 1]")
            print(f"  прогноз (норм. шкала): {y_n:.4f}")
            print(f"  прогноз (исх. шкала):  {y_hat:.4f}")
            if true is not None:
                print(f"  истинное x(k) = {true:.4f},  |ошибка| = {abs(true -
y_hat):.4f}")
            print()
            return y_n, y_hat, true
```

```
def _save_and_show(fig, name):
    fig.savefig(PLOTS_DIR / name, dpi=150, bbox_inches="tight")
    plt.show()
```

```
print(f"Вариант {VARIANT}: a={A}, b={B}, c={C}, d={D}, p={P_VAR}, N={N_VAR}")
print(f"alpha={ALPHA} (из диапазона 0.01–0.3), batch_size={BATCH_SIZE} (онлайн), "
      f"max_epochs={MAX_EPOCHS}, Ee={EE}")
print("Скорость 0.1: быстрее 0.01 и устойчивее 0.3 для сигмоиды в онлайн-режиме.")
```

6 Результаты экспериментов

6.1 Ряд, нормализация и окна

Сгенерированный ряд: 200 точек, min = −4,4995, max = 4,0511, размах = 8,5506. Нормализация по train: x_min = −4,4995, x_max = 4,0511. Тестовые значения в нормализованной шкале лежат в [0,0002; 0,9989], вне [0; 1] — 0 из 60 точек. В данном варианте экстремумы ряда попали на обучающий участок, поэтому тест формально остался внутри [0; 1]; правило «min/max только по train» при этом всё равно обязательно, иначе оценка была бы смещённой в общем случае.

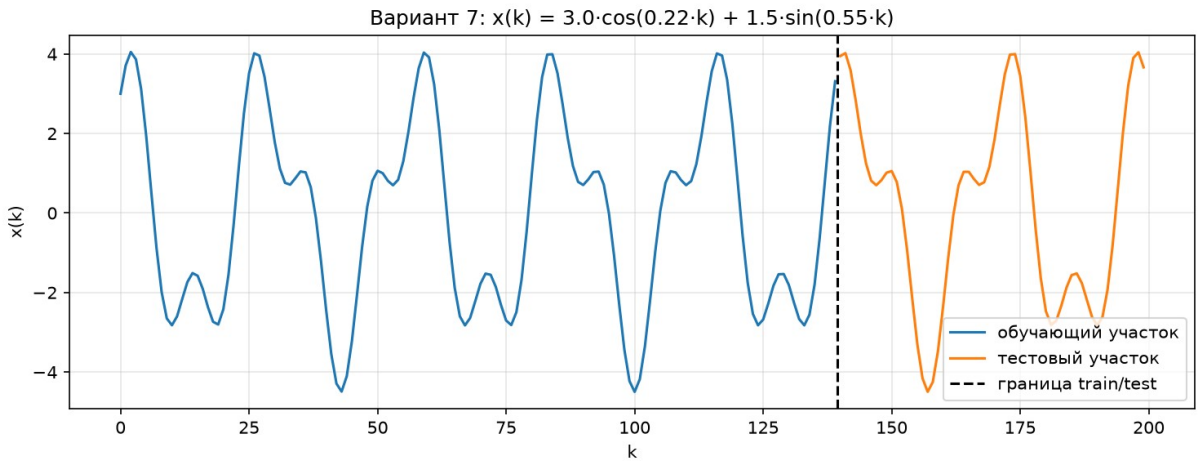


Рисунок 2 – Временной ряд варианта 7 и граница train/test

6.2 Базовый эксперимент

Конфигурация p = 8, N = 6, η = 0,1, seed = 42. Критерий E_s ≤ 10^{−4} достигнут за 185 эпох, итоговая E_s = 9,38·10^{−5}. Метрики в исходной шкале приведены в таблице 1. Train и test почти совпадают: сеть не переобучилась, ряд детерминированный.

Участок	MAE	RMSE	sMAPE, %
обучающий	0,0702	0,0828	8,54
тестовый	0,0692	0,0816	8,17

Таблица 1 – Метрики базовой сети 8–6–1 в исходной шкале

6.3 Исследование размера окна p

При фиксированном N = 6 обучены сети для p ∈ {2, 4, 8, 16}. По 5 запусков с seed ∈ {42, 7, 13, 21, 99}. Сводка — таблица 2.

p	Эпохи, ср. ± ст.	test MAE, ср. ± ст.	train MAE	не сошлось
2	20 000 ± 0	0,1772 ± 0,0018	0,1817	5 / 5
4	8494 ± 768	0,0669 ± 0,0007	0,0686	0 / 5
8	185 ± 13	0,0696 ± 0,0028	0,0700	0 / 5
16	24 ± 10	0,0614 ± 0,0063	0,0610	0 / 5

Таблица 2 – Влияние размера окна p при N = 6 (5 запусков)

При p = 2 ни один запуск не достиг E_e: окно из двух отсчётов не содержит достаточно фазовой информации о сумме двух гармоник. Уже при p = 4 ошибка падает в 2,6 раза, но обучение длится тысячи эпох. При p = 8 (вариант) ошибка почти такая же, как при p = 4, зато число эпох сокращается примерно в 45 раз. Дальнейшее увеличение до p = 16 даёт небольшой выигрыш по MAE (0,061 против 0,070) и ещё ускоряет сходимость, но разброс тестовой ошибки растёт. Практически ошибка перестаёт существенно уменьшаться после p = 4; выигрыш большего окна — в скорости, а не в принципиально новой точности.

6.4 Исследование числа скрытых нейронов N

При фиксированном p = 8 обучены сети для N ∈ {1, 2, 6, 12}, та же серия из 5 запусков (таблица 3).

N	Эпохи, ср. ± ст.	test MAE, ср. ± ст.	train MAE	не сошлось
1	211 ± 26	0,0683 ± 0,0019	0,0698	0 / 5
2	197 ± 8	0,0677 ± 0,0010	0,0694	0 / 5
6	185 ± 13	0,0696 ± 0,0028	0,0700	0 / 5
12	172 ± 13	0,0686 ± 0,0006	0,0706	0 / 5

Таблица 3 – Влияние числа скрытых нейронов N при p = 8

Минимального N = 1 уже достаточно, чтобы воспроизвести сумму двух гармоник в постановке «следующее значение по окну»: тестовый MAE совпадает с N = 6 и N = 12 в пределах разброса. Переобучения нет: train MAE и test MAE различаются

на сотые доли. Рост N слабо ускоряет обучение (211 → 172 эпохи) и почти не меняет точность. Для данного гладкого ряда избыточная ёмкость не вредна, но и не нужна.

6.5 Многошаговый прогноз

Лучшая по тестовому MAE конфигурация — p = 16, N = 6, seed = 21 (MAE = 0,0542). Рекурсивный прогноз на 30 шагов с k = 140: размах ряда 8,5506, порог 5 % равен 0,4275. Максимальная |ошибка| = 0,3082 на шаге 18, порог не превышен ни на одном шаге. Одношаговый прогноз на том же участке точнее, потому что в окно всегда попадают истинные значения; рекурсия подставляет собственные ошибки и накапливает их, что видно по росту |e| к середине горизонта и частичному спаду, когда ряд проходит более «знакомый» фрагмент периода.

Горизонт 30 шагов для лучшей сети можно считать пригодным целиком. Для базовой 8–6–1 одношаговая ошибка на тесте ≈ 0,07, то есть около 0,8 % размаха — существенно меньше рекурсивной.

6.6 Блок визуализации

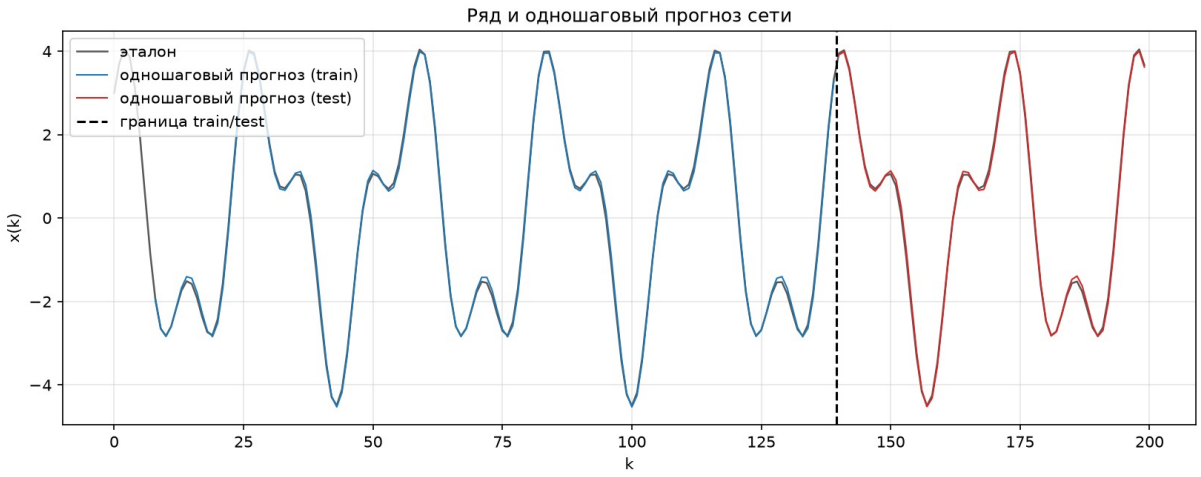


Рисунок 3 – Ряд и одношаговый прогноз сети 8–6–1

На рисунке 3 эталон и прогноз практически совпадают на обоих участках: это согласуется с близкими MAE train/test из таблицы 1.

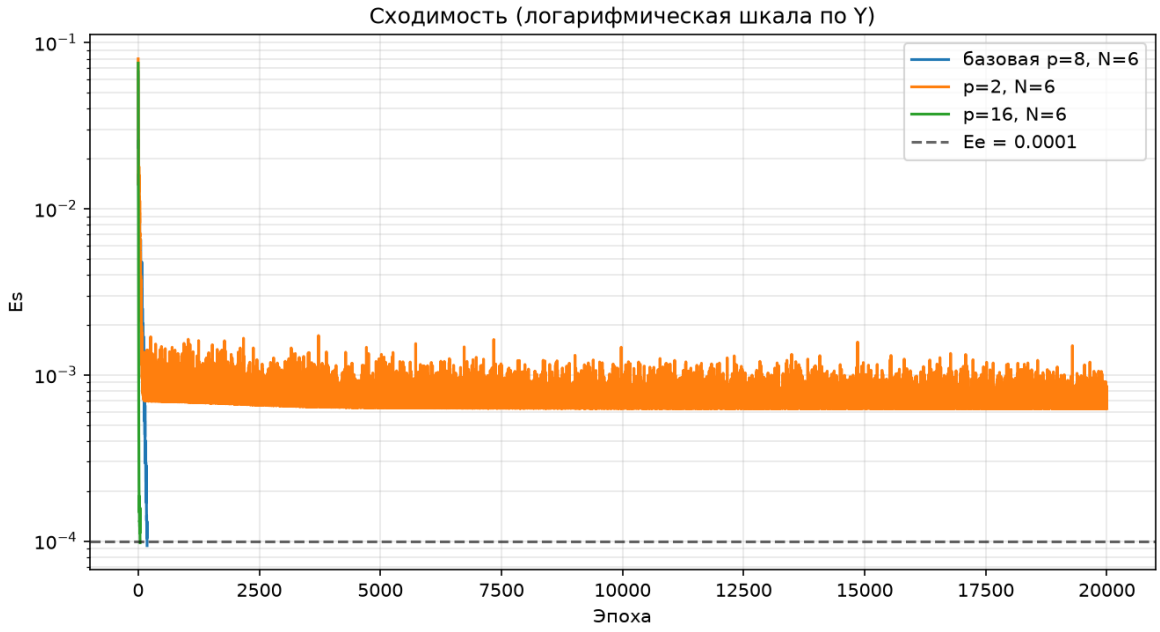


Рисунок 4 – Сходимость E_s (логарифмическая шкала по Y)

Базовая кривая (p = 8) и кривая p = 16 достигают E_e почти сразу в масштабе рисунка. Кривая p = 2 остаётся около 6·10^{−4} все 20 000 эпох и не пересекает уровень E_e — количественное подтверждение таблицы 2.

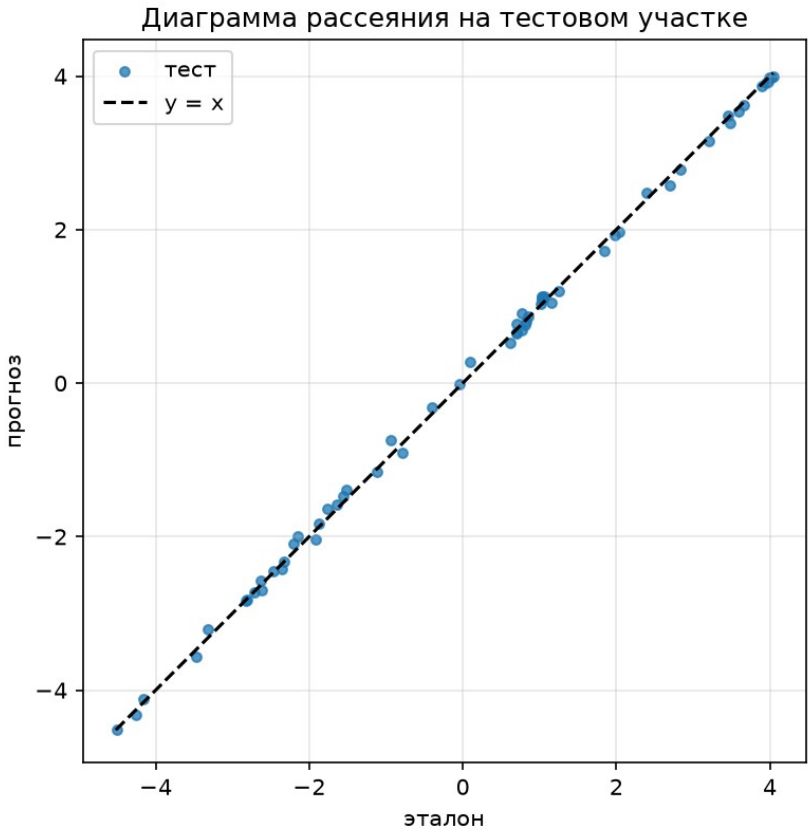


Рисунок 5 – Диаграмма рассеяния «эталон – прогноз» на тесте

Точки группируются вдоль диагонали y = x, систематического смещения нет.

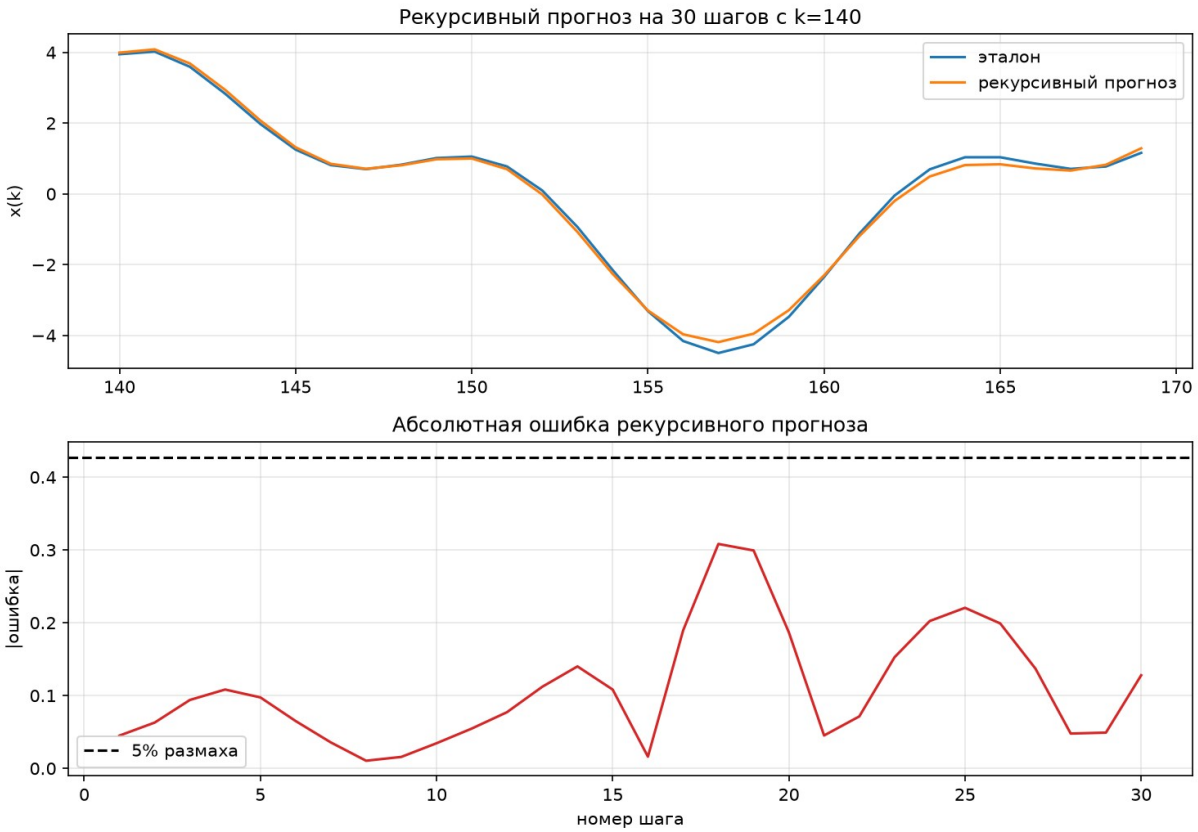


Рисунок 6 – Рекурсивный прогноз и рост абсолютной ошибки

6.7 Режим функционирования

Пользователь задаёт либо номер k, с которого берётся окно из ряда, либо p значений в исходной шкале. Программа выводит нормализованные входы, прогноз в нормализованной шкале (4 знака), прогноз в исходной шкале и, если окно взято из ряда, истинное x(k) и абсолютную ошибку. Результаты для базовой сети 8–6–1 — таблица 4.

Окно	ŷ норм.	ŷ исх.	x(k)	|ошибка|	примечание
train, k=20	0,2319	−2,5166	−2,4220	0,0946	
train, k=70	0,3245	−1,7249	−1,7819	0,0570	
train, k=120	0,6382	0,9578	0,8569	0,1009	
test, k=145	0,6662	1,1971	1,2518	0,0547	
test, k=170	0,7274	1,7200	1,8468	0,1268	

test, k=195	0,7568	1,9719	2,0412	0,0693	
польз. фрагмент	0,5334	0,0613	—	—	ряд [30:38]
выброс +4	0,3418	−1,5770	1,0554	2,6324	вход вне [0;1]

Таблица 4 – Режим функционирования сети 8–6–1

На «чистых» окнах ошибка того же порядка, что MAE теста (0,05–0,13). Пользовательское окно с выбросом +4 в середине даёт нормализованный вход 1,089 > 1 и срывает прогноз (ошибка 2,63). Это ожидаемо: сеть обучена на гладком ряде в диапазоне train и не обязана экстраполировать аномалию.

6.8 Эксперимент с шумом

На обучающий участок добавлен гауссов шум σ = 0,02·(x_max − x_min) = 0,171. Оценка — на чистом ряде (таблица 5). По сравнению с таблицей 1 тестовый MAE вырос с 0,069 до 0,125 (N = 6) и 0,121 (N = 12). Удвоение N не ухудшило обобщение: сеть не запомнила шум. Разрыв «шумный train — чистый test» объясняется несглаженной целью, а не переобучением.

N	MAE на шумном train	MAE на чистом train	MAE на чистом test
6	0,2687	0,1250	0,1252
12	0,2634	0,1205	0,1210

Таблица 5 – Обучение на зашумлённом ряде, оценка на чистом

7 Выводы

Ниже даны ответы на вопросы лабораторных работ № 1–3 и вопросы лабораторной работы № 4. Ответы опираются на фактические результаты соответствующих экспериментов.

7.1 Вопросы лабораторных работ № 1–3

Лабораторная работа № 1. *Почему однослойный персептрон не решает XOR и зачем нужен скрытый слой?* XOR линейно неразделим: одна гиперплоскость не отделяет классы (c0,c0) и (c1,c1) от (c0,c1) и (c1,c0). В ЛР № 1 однослойная сеть застряла около E_s = 0,5, тогда как MLP 2–2–1 обучился. Скрытый слой строит нелинейные признаки, после которых задача становится линейно разделимой на выходном нейроне.

Лабораторная работа № 2. *Как онлайн-режим и случайная инициализация влияют на обучение?* Онлайн-режим (batch_size = 1) обновляет веса после каждого примера и при перемешивании снижает риск застревания в одном порядке подачи. Пять разных seed в ЛР № 2–4 дают разброс числа эпох при почти одинаковой итоговой ошибке: инициализация меняет траекторию, но не обязательно качество, если задача простая и η подобрана.

Лабораторная работа № 3. *Почему на выходе классификатора сигмоида и зачем пересчёт в [c0; c1]?* Сигмоида отображает выход в (0; 1) и согласована с метками 0/1 и с BCE. Исходные уровни варианта 7 (c0 = −4, c1 = −9) восстанавливаются линейным пересчётом ŷ_real = c0 + ŷ(c1 − c0). Если обучать сигмоиду сразу на ненормализованных −4/−9, насыщение производной остановит обучение. ReLU в скрытом слое ускоряет прямое распространение, но на четырёх точках XOR выигрыш невелик по сравнению с сигмоидой.

7.2 Вопросы лабораторной работы № 4

Как размер окна p влияет на качество? При каком p ошибка перестаёт уменьшаться? По таблице 2 ошибка резко падает от p = 2 (MAE ≈ 0,18, 5/5 не сошлись) к p = 4 (MAE ≈ 0,067). Дальше, при p = 8 и p = 16, MAE меняется слабо (0,070 и 0,061). Существенное уменьшение ошибки прекращается после p = 4; большее окно в основном ускоряет сходжение к E_e.

Почему p = 2 не описывает сумму двух гармоник, а слишком большое окно не даёт выигрыша. Ряд — сумма двух частот; состояние осциллятора определяется амплитудой и фазой каждой гармоники, то есть не менее чем четырьмя степенями свободы. Двух отсчётов недостаточно, чтобы однозначно восстановить обе фазы, поэтому сеть с p = 2 аппроксимирует грубое среднее и не доходит до E_e. Слишком большое p (16) содержит избыточную, сильно коррелированную историю: новые признаки почти линейно зависят от уже имеющихся, поэтому MAE почти не улучшается, растёт лишь разброс между запусками.

Как N влияет на точность и скорость? Есть ли переобучение? Достаточно ли минимального N? По таблице 3 точность от N почти не зависит, число эпох слабо убывает (211 → 172). Переобучения нет: test MAE ≤ train MAE. Минимального N = 1 достаточно для данной гладкой суммы гармоник в постановке одношагового прогноза по окну.

Почему на выходе линейная функция, а не сигмоида? Прогноз — вещественное число в исходной шкале после

денормализации, а в нормализованной шкале цель ∈ [0; 1], но сеть должна иметь право выйти за этот отрезок, если тест шире train. Сигмоида принудительно сожмёт выход в (0; 1). Если к тому же обучать её на ненормализованных x_k ∈ [−4,5; 4,1], аргумент попадёт в область насыщения, градиент обнулится, и обучение остановится — ровно та проблема, которая в ЛР № 3 решалась переводом меток в 0/1.

Почему нормализация только по train? Что будет, если тест выйдет из [0; 1]? Иначе масштаб «подглядывает» в будущее. Выход теста за [0; 1] означает, что появились амплитуды, которых сеть не видела; линейный выход их формально предскажет, но ошибка вырастет. На наших данных тест остался в [0,0002; 0,9989]: экстремумы ряда попали в train. Это удача варианта, а не основание брать min/max по всему ряду.

Насколько устойчиво обучение к инициализации? При p ≥ 4 и любом исследованном N все 5 запусков сошлись. Разброс эпох мал для p = 8 (185 ± 13) и заметен для p = 4 (8494 ± 768) и p = 16 (24 ± 10). Не достигшие E_e запуски были только при p = 2 (все пять) — причина в недостатке признаков, а не в η или N. При выбранном η = 0,1 обучение устойчиво.

Как растёт ошибка рекурсивного прогноза? Ошибка накапливается, потому что искажённое ŷ сдвигает следующее окно. На рисунке 6 она колеблется и достигает максимума 0,308 на 18-м шаге, не пересекая 5 % размаха (0,428). Одношаговый прогноз точнее (MAE ≈ 0,05–0,07). Для лучшей сети пригодны все

30 шагов; разумный рабочий горизонт — не меньше периода основной гармоники, пока |e| держится ниже порога.

Как различие train/test связано с обобщением? Что дал шум? На чистом ряде MAE train и test совпадают — обобщающая способность высокая, выборка и тест принадлежат одному детерминированному процессу. Меры улучшения обобщения (ранняя остановка, уменьшение N, шум, рост выборки) здесь почти не нужны: N = 1 уже не переобучается. Эксперимент с шумом увеличил test MAE примерно в 1,8 раза (0,069 → 0,125), но разрыв train/test на чистом ряде остался нулевым, а N = 12 не запомнил шум лучше, чем N = 6. Шум снизил точность как регуляризатор «вширь», а не вызвал классическое переобучение.

Таким образом, для варианта 7 рабочая конфигурация p = 8, N = 6 обоснована: она быстро сходится, устойчива к seed и не переобучается. Если важна скорость — достаточно p = 16; если важна интерпретируемость ёмкости — достаточно N = 1 при p = 8.